



ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

ROS - REPORT

EPFL COURSEWORK

Melvyn Aldwin Truc-Lâm Huynh
Florian Philippe Julien Kighelman
Yann Martin Thomé

April 17, 2026

Contents

1	Introduction	2
2	Session 1: ROS2 Fundamentals	3
2.1	Workspace, Packages & First Node	3
2.2	Topics: Publishers & Subscribers	3
2.3	Services & Custom Interfaces	3
3	Session 2: URDF, RViz & Gazebo	5
3.1	RViz, TFs & Launch Files	5
3.2	URDF & Xacro	5
3.3	Building the Thymio Model (Steps 1–3)	5
3.4	Gazebo Integration (Steps 4–6)	5
4	Session 3: Thymio Mini-Project	7
4.1	Project Description	7
4.2	Implementation	7
4.3	Results	8
5	Reflections on ROS2 Key Concepts	9
5.1	The Computation Graph & Communication Patterns	9
5.2	URDF, Transforms & Simulation	9
5.3	Debugging as a Core Skill	9
6	Conclusion	10

Chapter 1

Introduction

This report documents our work through the three sessions of the MICRO-453 ROS2 Basics course, along with the preparatory exercises that preceded each one. ROS2 is the middleware framework that most modern robotics projects are built on: it provides a way to split complex robot behaviour into independent processes (nodes) that communicate through well-defined channels. The course gave us a hands-on introduction to this ecosystem, starting from scratch and building up to an autonomous maze-navigation controller.

We began in Session 1 with the basics: setting up the Docker environment, creating workspaces, writing nodes, and understanding how topics and services work as communication patterns. Session 2 moved into robot modelling — we built the Thymio’s URDF step by step using Xacro, visualised it in RViz, and got it running in the Gazebo simulator with a differential drive plugin and proximity sensors. Session 3 brought everything together in a mini-project where we programmed the Thymio to navigate a two-part maze autonomously, which required combining velocity commands, sensor readings, odometry-based turning, and asynchronous service calls for entity management.

For each session we describe what we implemented, the problems we ran into and how we fixed them, and what we learned. A lot of the issues were small but instructive, forgetting to source the workspace, missing collision tags, stale odometry after a respawn, and solving them often taught us more than the exercises themselves. We close with reflections on the key ROS2 concepts and a short conclusion.

Chapter 2

Session 1: ROS2 Fundamentals

2.1 Workspace, Packages & First Node

We verified the ROS2 installation with the demo `talker/listener` nodes and learned the role of `source /opt/ros/humble/setup.bash` in `.bashrc`: without it, ROS2 commands are unavailable. We then created a workspace (`ros2_basics_ws`), a Python package (`ros2_basics_pkg`), and built it with `colcon build`. The build generates `/log`, `/build`, and `/install` directories; only `/install` is used by ROS2 at runtime.

Our first node (`minimal_node.py`) used `rclpy`: a class inheriting from `Node`, registered via `super().__init__("node_name")`, and kept alive by `rclpy.spin()`. Commenting out `spin()` confirmed that without it the node exits immediately. Executables are declared in `setup.py` via `entry_points`.

Issue: Node not found after build

`ros2 run` could not find the node. **Solution:** We had forgotten to source the workspace. `source install/setup.bash` resolved it. This was a recurring issue throughout the course.

2.2 Topics: Publishers & Subscribers

Topics provide asynchronous, many-to-many publish/subscribe communication. We created a publisher (`create_publisher(String, 'topic', 10)` with a timer callback) and subscriber (`create_subscription(String, 'topic', callback, 10)`), and debugged using `ros2 topic list/echo`, `ros2 node info`, and `rqt_graph`. We practiced remapping nodes (`-ros-args -r __node:=new_name`) and topics at runtime.

Exercise 1 required a Heat Index Monitoring System: two publishers simulating temperature (27–43°C) and humidity (40–100%) sensors, and a subscriber computing the heat index and logging warnings above 125°F. This reinforced multi-topic subscription and data fusion in a single node.

2.3 Services & Custom Interfaces

Services provide synchronous request/response communication. We implemented the `add_two_ints` pattern with a server (`create_service()`) and an asynchronous client (`call_async()` with `functools.partial`). Starting the client before the server showed it waiting for service discovery.

For custom interfaces, we created a C++ package (`ros2_basics_interfaces`) with `msg/` and `srv/` directories. The `CMiEquipmentStatus.msg` example included fields like `string equipment_id`, `float32 temperature`, `bool is_operational`. Proper configuration of `CMakeLists.txt` `rosidl_generate_interfaces` is required.

Exercise 2 (Prep Work 2) extended this with a `MissionValidation.srv` service for a rover, validating temperature, battery, and distance conditions.

Issue: Custom message import fails

ModuleNotFoundError after building the interfaces package. **Solution:** Re-source the workspace with `source install/setup.bash` to make generated Python bindings available.

Key Concepts — Session 1

Nodes: independent processes. **Topics:** async pub/sub for continuous data. **Services:** sync request/response for discrete operations. **Custom interfaces:** `.msg/.srv` in a C++ package. **Debugging:** `ros2 topic/node/service` CLI and `rqt_graph`.

Chapter 3

Session 2: URDF, RViz & Gazebo

3.1 RViz, TFs & Launch Files

We explored the `urdf_tutorial` package in RViz, learning about **Transform Frames (TFs)**, coordinate frames defining spatial parent–child relationships between robot parts. The `tf2_tools view_frames` command generates a PDF of the TF tree with `base_link` as root. The Joint State Publisher GUI allowed interactive joint manipulation.

Launch files (XML format) start multiple nodes simultaneously with parameters and remappings. We used them throughout the session to launch RViz, `robot_state_publisher`, and Gazebo with a single command.

3.2 URDF & Xacro

A URDF describes a robot using **links** (rigid bodies with visual/collision/inertial properties) and **joints** (fixed, revolute, continuous, or prismatic connections). We learned the critical **origin placement procedure**: (1) zero all origins, (2) position the parent visual, (3) set the joint origin relative to the parent, (4) orient and position the child visual. The joint origin is the most important as it defines the TFs.

Xacro extends URDF with variables (`<xacro:property>`), math expressions (`/${width}/2.0`), macros (`<xacro:macro>`), and file inclusion — eliminating hardcoded values.

3.3 Building the Thymio Model (Steps 1–3)

We built the Thymio URDF progressively:

Component	Geometry	Colour	Joint
Base link	Box: $11 \times 11.2 \times 4.4$ cm	White	—
Caster wheel	Sphere: $r = 9$ mm	White	Fixed
Wheels (L/R)	Cylinder: $l = 1.5$ cm, $r = 2.2$ cm	Black	Continuous

In **Step 2**, we refactored into three Xacro files: `materials.xacro`, `thymio_chassis.xacro` (with a wheel macro reused for both sides), and `thymio.urdf.xacro`. In **Step 3**, we added `<collision>` and `<inertial>` tags with reusable inertia macros. Mass distribution: 270 g total (80% chassis, 20% wheels).

Issue: Robot falls through ground in Gazebo

Missing `<collision>` tags on wheel links. **Solution:** Add collision geometry to every link. We also found that omitting `<inertial>` tags makes links unresponsive to physics.

3.4 Gazebo Integration (Steps 4–6)

To fix unwanted drift after spawning, we added joint dynamics (`damping="0.1"`, `friction="0.2"`) and reduced caster friction (`<mu1>/<mu2> = 0.31`) in a separate `gazebo.xacro` file. Gazebo materials were also defined there, since URDF `<material>` tags are ignored by Gazebo.

For control, we configured the `gazebo_ros_diff_drive` plugin specifying wheel joints, separa-

tion, diameter, `cmd_vel` input, and odometry output. We tested with `ros2 topic pub /cmd_vel geometry_msgs/msg/Twist`. Finally, proximity sensors (front/rear) were integrated from a provided `proximity_sensors.xacro`, adjusting the base link length to 9.1 cm for proper sensor placement.

Issue: Backward drift

The Thymio deviates when moving backward. **Solution:** Known limitation of the simplified model. We avoided backward commands in our controller.

Key Concepts — Session 2

URDF/Xacro: parametric robot description with visual, collision, and inertial properties. **Tfs:** auto-generated from joints; critical for all ROS2 spatial reasoning. **Gazebo:** physics simulation requiring plugins for actuation and sensing.

Chapter 4

Session 3: Thymio Mini-Project

4.1 Project Description

The goal of this mini-project was to guide a simulated Thymio through a two-part Gazebo maze. We had to apply the ROS2 concepts seen in the previous sessions: sending velocity commands via topics, reading proximity sensors to detect walls, and using Gazebo services to manage the robot entity. The `thymio_mini_project` package was provided and already included the Thymio model with its differential drive controller and proximity sensors, as well as the maze world. Each turn in the maze had only one possible direction, always indicated by a wall directly ahead, which made the logic straightforward. At dead ends, we either teleported the robot to the second section or stopped it definitively.

4.2 Implementation

We implemented the controller as a single `ThymioController` node. To manage the different phases of the task, we used a state machine with six states: `phase1`, `deleting`, `spawning`, `phase2`, `turning`, and `stopped`. A timer at 10 Hz drives the main loop, and we publish a `Twist` to `/cmd_vel` on every tick, including a zero command when the robot is not supposed to move.

Forward motion and wall detection. We wrote a `move_until_wall()` method that reads the distance from the front proximity sensor, which arrives as a `LaserScan` on thymio front center proximity sensor. When there is no obstacle in range, the sensor returns `inf`, so we initialised `self.dist_` to `float('inf')` to handle this case. We then applied a progressive speed reduction as the robot gets closer to the wall: 0.1 m/s far away, 0.05 m/s within 25 cm, and 0.03 m/s very close to the stop threshold. We also used two different stop distances: 1.5 cm before a turn so that the robot is well-centred in the corridor, and 5 cm at the final dead ends.

Odometry-based turning. For the turns, we wrote a `turn_angle()` method that uses odometry data from `/odom`. We extract the yaw at each message using `euler_from_quaternion()`. We quickly realised that computing each target angle as a relative increment from the previous heading was a bad idea, as errors would accumulate over the four turns. Instead, we referenced all targets to the known spawn yaw of $\pi/2$ using the formula:

$$\psi^* = \text{wrap_angle}\left(\psi_{\text{spawn}} + (n + 1) \frac{\pi}{2}\right)$$

where n is the number of completed turns. This way, each target is independent and no drift builds up. We stop the rotation when the error falls below 0.01 rad. We also added a 20 cm minimum travel guard after each turn to prevent the wall-detection from triggering too early before the robot has fully entered the next corridor.

Teleportation between maze sections. Once the Thymio stopped at the first dead end, we called `/delete_entity` asynchronously. Then, directly inside the `on_deleted()` callback, we sent the `/spawn_entity` request with the URDF we generated at startup using `xacro.process_file()`. We placed the new entity at $(x = 0.5, y = 0.3, \text{yaw} = \pi/2)$ with the right quaternion from `quaternion_from_euler(0, 0, \pi/2)`. Putting the spawn directly inside the delete callback was the cleanest way we found to guarantee the correct order of operations without using artificial delays.

Issue: Robot disappearing after teleportation

After the delete/spawn sequence, the Thymio would move a few centimetres and then suddenly disappear. We added a lot of `get_logger().info()` prints throughout the callbacks to understand what was happening. We found that the controller was reading stale odometry data from the old deleted entity right after the spawn, which caused it to trigger a second unintended delete. **Solution:** We added an `odom_ready_` flag that we reset to `False` on every spawn. The node stays in `spawning` until the first fresh odometry message arrives from the new entity. Only then do we record `start_pos_` and transition to `phase2`, which ensures we never act on leftover data from the deleted robot.

Issue: Imprecise turns

In our first attempts, we accumulated relative angle increments from one turn to the next. After a few turns, the heading drift was large enough to misalign the robot with the corridor. **Solution:** We switched to absolute target headings computed from the spawn yaw as described above. We also tuned the proportional speed controller and set a minimum angular speed of 0.03 rad/s to avoid stalling when the error gets very small. This combination solved the alignment issue reliably.

4.3 Results

The Thymio successfully completed both maze sections across several runs. The graduated speed profile made stopping smooth and precise, the absolute-yaw strategy kept all four turns accurate, and the `odom_ready_` flag made the teleportation robust. We also benefited from only using forward motion and in-place rotations, which meant we never triggered the backward-drift issue we had observed in Session 2. The trickiest part was tuning the intermediate stopping distance of 1.5 cm: if we set it too large the robot stopped too early and missed the corridor centre after turning, and if we set it too small it risked bumping into the wall. The value we settled on worked well throughout all our tests.

Project Outcome

The project was successful. We managed to guide the Thymio through both maze sections autonomously and reliably. The two decisions that made the biggest difference were using absolute yaw references to avoid turn drift and introducing the `odom_ready_` flag to fix the teleportation bug. The main challenge was getting the stopping distance right, as it directly affected how well the robot aligned with each corridor after turning.

Chapter 5

Reflections on ROS2 Key Concepts

5.1 The Computation Graph & Communication Patterns

ROS2’s modular architecture connects nodes through topics and services. The Thymio project overview from Prep Work 2 illustrated this clearly: sensor, behaviour, and control nodes forming a graph with well-defined data flows. The key design decision is choosing between **topics** (continuous async data: sensor streams, velocity commands, odometry) and **services** (discrete sync operations: entity management, configuration changes). The mini-project made this concrete: `cmd_vel` and sensor data used topics; entity deletion/spawn used services.

5.2 URDF, Transforms & Simulation

A robot model is fundamentally a **tree of coordinate frames**. The `robot_state_publisher` broadcasts TFs from the URDF and joint states, and every downstream component depends on these being correct. Xacro’s parameterisation and macros were essential for maintainability — changing the base width propagated automatically to wheel positions.

Gazebo enabled rapid iteration without hardware, but the “sim-to-real gap” was evident: friction tuning, backward drift, and simplified inertia required workarounds. The integration pipeline (URDF → `robot_state_publisher` → TFs → Gazebo plugins) is foundational knowledge for any ROS2 project.

5.3 Debugging as a Core Skill

CLI tools (`ros2 topic list -t`, `echo`, `ros2 node info`, `rqt_graph`) became indispensable. Publishing test messages from the terminal (`ros2 topic pub`) allowed testing subscribers in isolation. These tools turned opaque failures into diagnosable problems.

Chapter 6

Conclusion

This course provided a hands-on introduction to ROS2 through a well-structured progression: environment setup, core concepts (nodes, topics, services, interfaces), robot modelling (URDF/Xacro), simulation (RViz, Gazebo), and a complete mini-project. The main challenges, from workspace sourcing, URDF origin placement to asynchronous service coordination. Each reinforced fundamental concepts. We are now confident in building ROS2 packages, implementing communication patterns, designing parametric robot models, and building autonomous controllers.